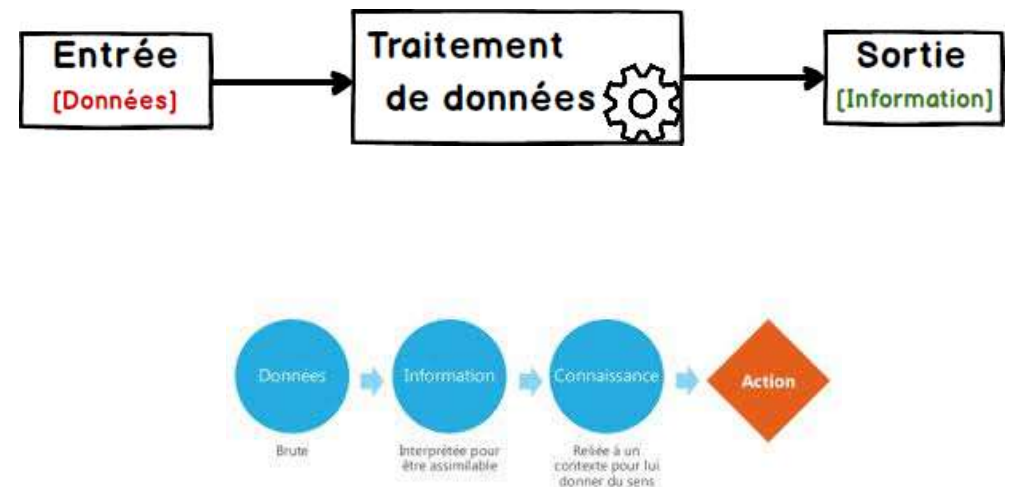
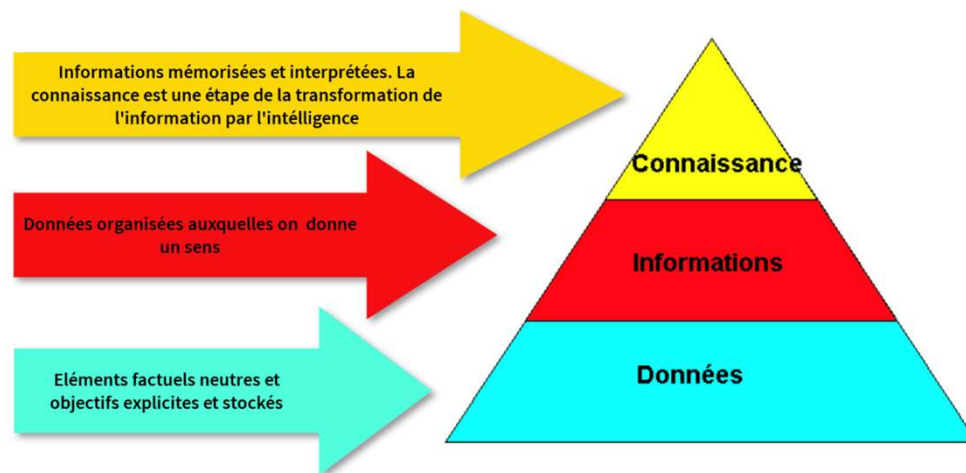


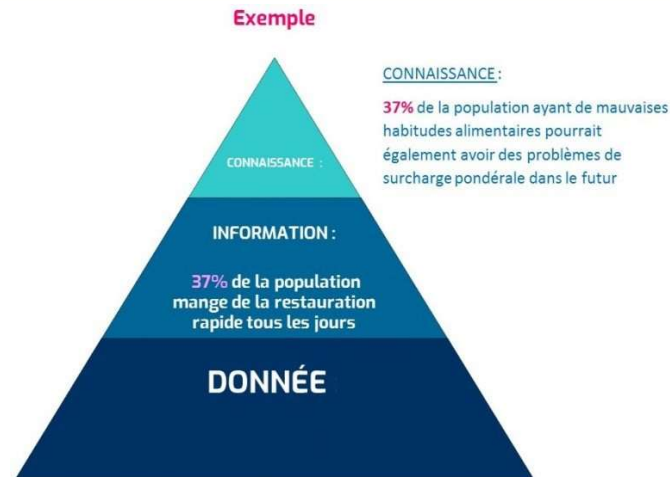
Chapitre 01_Introduction à l'Administration des Bases de Données

Définition d'une base de données

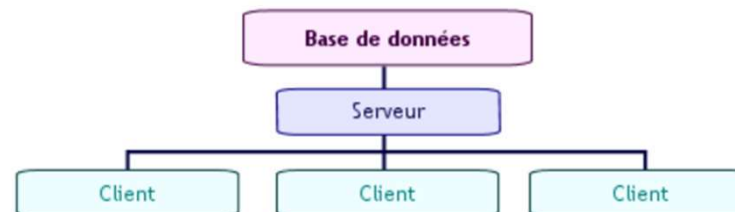
- ❑ Le terme de base de données (en anglais « **DB** » signifie *data base*) correspond à une **entité dans laquelle il est possible de stocker des données de façon structurée et avec le moins de redondance possible**. Ces données doivent pouvoir être utilisées par des programmes, par des utilisateurs différents.
- ❑ **Différence entre informations et données** : Les **données** sont un ensemble de **faits bruts**, de chiffres et de statistiques qui sont **traités** pour en **extraire des informations significatives**. En revanche, l'information est le résultat de l'**analyse** et de l'**interprétation** des données pour générer des idées et des **connaissances**. En d'autres termes, les **données** sont l'**entrée**, tandis que l'**information** est la **sortie**. Les **bases de données** stockent des données qui peuvent être utilisées pour générer des informations.



Définition d'une base de données



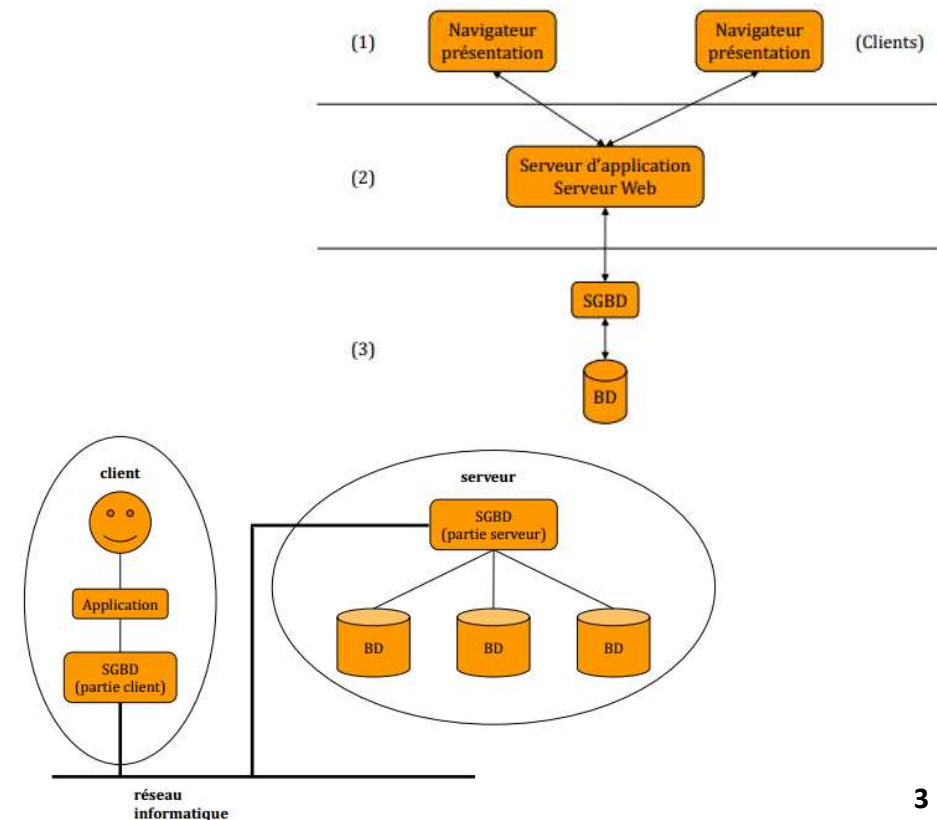
- ❑ Ainsi, la notion de base de données est généralement couplée à celle de **réseau**, afin de pouvoir **mettre en commun** ces données, d'où le **nom de base**. On parle généralement de **système d'information** pour désigner toute la structure regroupant les moyens mis en place pour pouvoir partager des données.



- ❑ **BDD** désignent une **collection de données organisées** pour être facilement consultables, gérables et mises à jour. Ainsi, au sein d'une database, les données sont soumises à une **organisation rigoureuse** : en **ligne**, **colonnes** et **tableaux**.

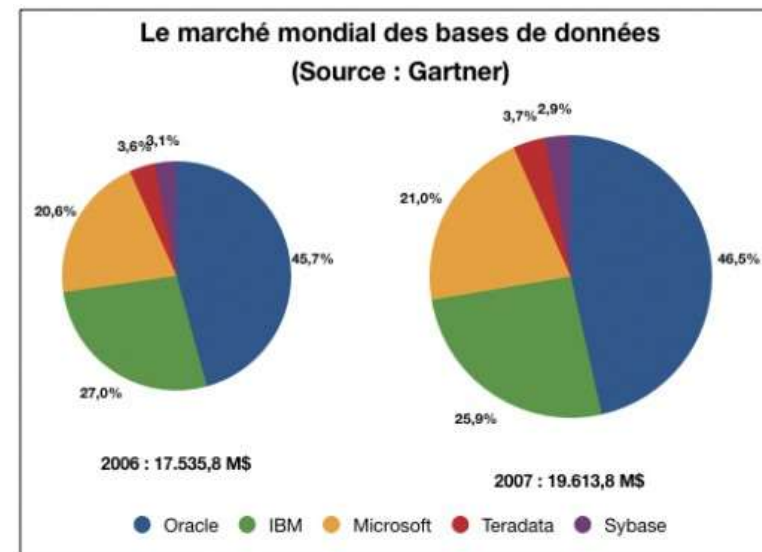
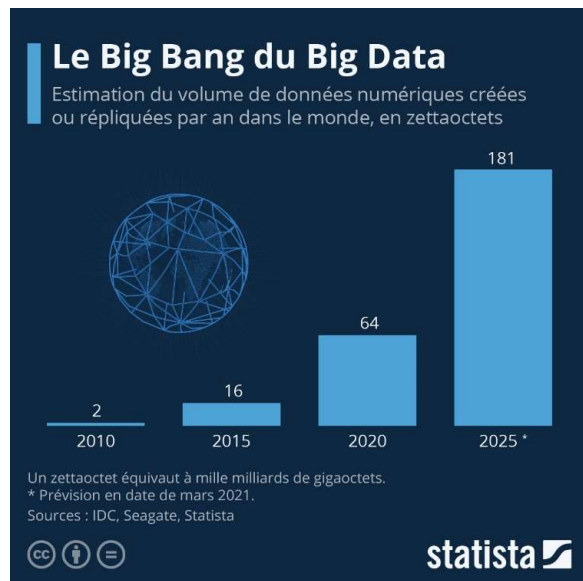
Définition d'une base de données

- ❑ Elles sont **indexées** pour être trouvées **rapidement** via un logiciel informatique, et mises à jour ou supprimées à chaque fois que de nouvelles informations sont ajoutées.
- ❑ Afin de pouvoir contrôler les données ainsi que les utilisateurs, le besoin d'un **système de gestion** s'est vite fait ressentir. La gestion de la base de données se fait grâce à un système appelé **SGBD** (système de gestion de bases de données) ou en anglais **DBMS** (*Data base management system*).
- ❑ Le **SGBD** est un **ensemble de services** (applications logicielles) permettant de gérer les bases de données, c'est-à-dire permettre **l'accès aux données** de façon simple, autoriser un **accès** aux informations à de **multiples utilisateurs** et **manipuler** les données présentes dans la base de données (insertion, suppression, modification).
- ❑ Ce SGBD est décrit à l'aide d'un **langage qui permet de définir les différents objets qui la composent**. De plus, ce système d'information doit fournir aux acteurs de cette organisation les **services** qui leur permettront d'**extraire** les informations **pertinentes** mais aussi de les mettre à jour. C'est ce que réalise le **système SQL**.



Définition d'une base de données

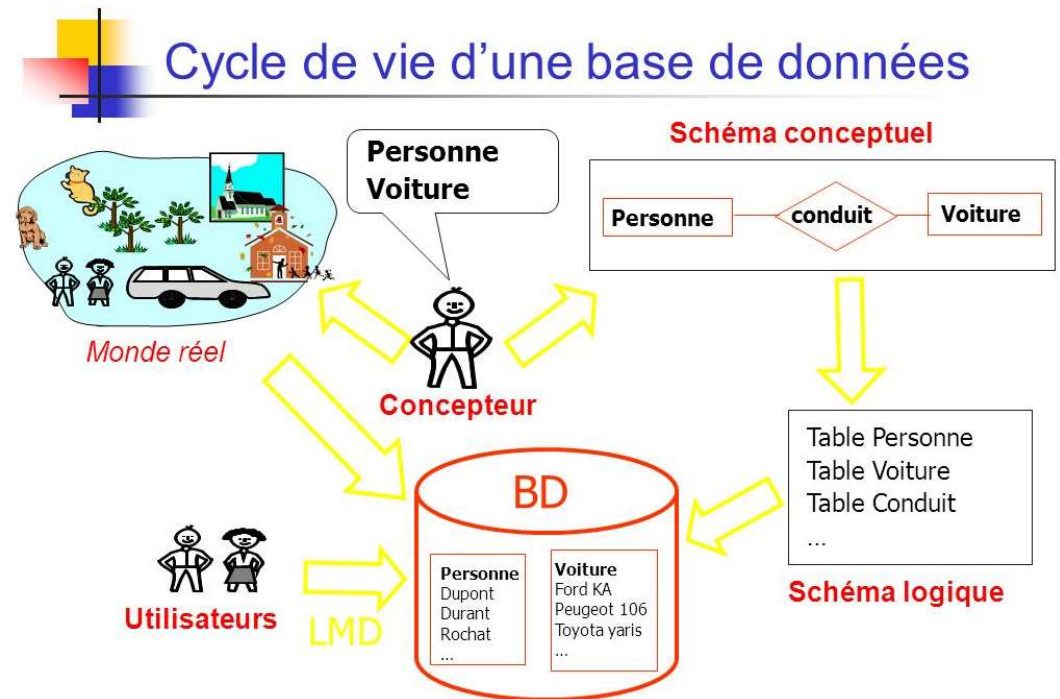
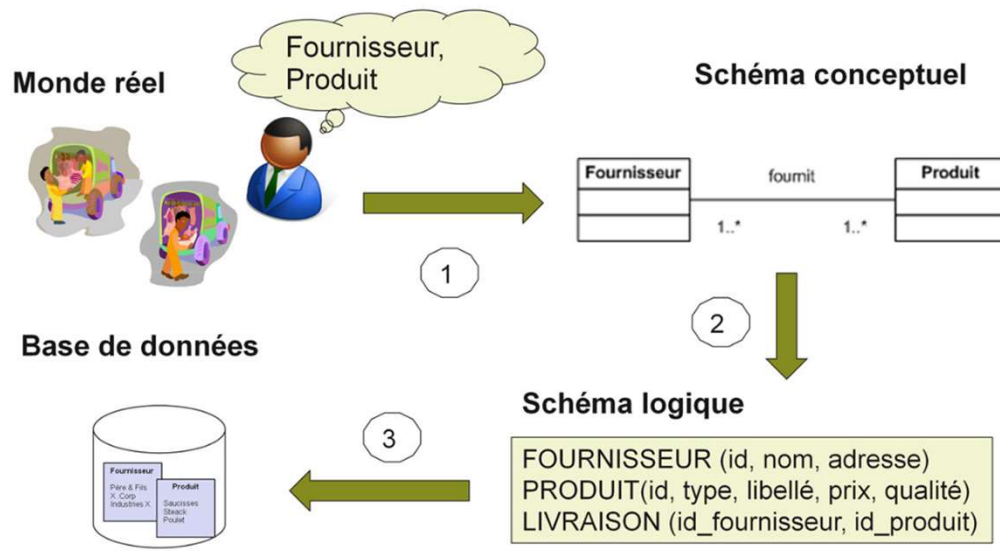
- ❑ **L'utilité d'une base donnée** est de **mettre des données à la disposition** d'utilisateurs pour une consultation, une saisie ou bien une mise à jour, tout en s'assurant des **droits accordés** à ces derniers. Cela est d'autant **plus utile** que les données informatiques sont de **plus en plus nombreuses**. L'avantage majeur d'une base de données hormis de regrouper un grand nombre d'informations demeure la **possibilité d'y accéder par plusieurs utilisateurs simultanément**.
- ❑ Les bases de données sont utilisées par de nombreuses entreprises dans toutes les industries. Elles sont notamment utilisées par les **compagnies aériennes pour gérer les réservations**. Elles sont utilisées pour la **gestion de production**. Pour les **enregistrements médicaux dans les hôpitaux**, ou encore pour les **enregistrements légaux dans les compagnies d'assurances**. Les bases de données les plus larges sont généralement utilisées **par les agences gouvernementales**, les grandes entreprises ou les universités.



Modélisation d'une base de données

« Une base de données est un ensemble structuré et cohérent de données enregistrées avec le minimum de redondance pour satisfaire simultanément plusieurs utilisateurs de manière sélective et dans un temps opportun. »

Une base de données représente un microcosme, c'est à dire une partie du monde.





Les objectifs d'une base de données

Les bases de données ont été conçues pour répondre aux 4 objectifs suivants :

Intégration et corrélation :

- A l'origine chaque programme disposait de ses propres données, d'où une **forte redondance** des informations.
- Le problème majeur était de **garantir la cohérence** de ces informations entre les systèmes.
- Le but était ainsi de **centraliser** les données pour **éviter la redondance** des données (**gagner** ainsi de l'espace disque), et d'assurer la cohérence des données.

Flexibilité et indépendance :

- La base de données est censée assurer trois niveaux d'indépendance :
 - **Indépendance Physique** : indépendance des données vis à vis du matériel utilisé.
 - **Indépendance Logique** : indépendance des données vis-à-vis des schémas et sous schémas utilisés pour représenter les données.
 - **Indépendance d'Accès** : les méthodes d'accès aux données sont désormais gérées par le **SGBD**. (accès direct, accès séquentiel, indexation, pointeurs).

Disponibilité :

- La base de donnée permet de gérer la **concurrence d'accès**, de modification et de consultation des données. Cela afin d'**améliorer** le temps de réponse.

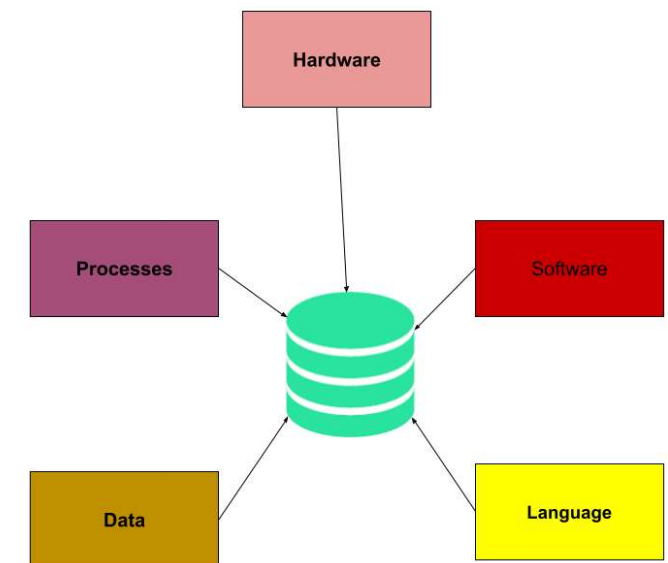
Sécurité :

- La base de données a pour but de garantir l'**intégrité** et la **confidentialité** des données.

Les 5 composants d'une base de données

Une base de données se compose de **cinq éléments principaux** : le **matériel**, le **logiciel**, les **données**, la **procédure** et le **langage** d'accès à la base de données.

- Le **matériel (hardware)** est constitué d'appareils électroniques tels que des ordinateurs et des systèmes de stockage.
- La **partie logicielle** est un ensemble de programmes utilisés pour gérer et contrôler la base de données. Elle regroupe le **logiciel de base de données (SGBD)**, le système d'exploitation, le logiciel de réseau permettant le partage des données, et les applications permettent l'accès aux données.
- Les **données** sont les **informations brutes** qui sont stockées dans la base de données. Elles peuvent prendre divers formats, tels que des textes, des images ou des vidéos.
- La **procédure** est un **ensemble de règles** et d'instructions permettant d'utiliser le logiciel de gestion de base de données (DBMS/SGBD).
- Enfin, le **langage (ex: SQL)** d'accès à la base de données est utilisé pour accéder aux données, pour en entrer de nouvelles, pour mettre à jour les données existantes ou pour les récupérer via les requêtes.



Les composants d'un SGBD





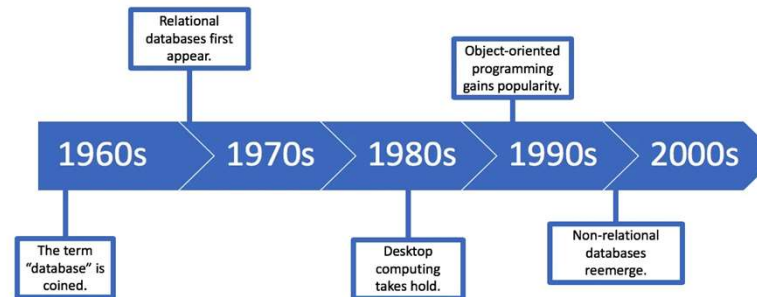
Historique et types de base de données

- L'histoire des bases de données **remonte aux années 1960**, avec l'apparition des **bases de données réseau** et des **bases de données hiérarchiques**. Les **bases de données relationnelles** ont été inventées en **1970** par E.F. Codd de IBM.
- Dans les années **1980**, ce sont les **bases de données object-oriented** qui ont fait leur apparition. Aujourd'hui, les bases de données prédominantes sont les **bases relationnelles (SQL)**, bases **NoSQL** et **bases de données cloud**.
- Il est aussi possible de classer les bases de données en **fonction de leur contenu** : bibliographique, textes, nombres ou images.
- Toutefois, en informatique, on classe généralement les bases de données en fonction de leur **approche organisationnelle**. Il existe de nombreux types de bases de données différentes : **relationnelle, distribuée, cloud, NoSQL...** Ci-après les différents types de bases de données.

Dates phases de BD

- 1960 : Apparition des premières bases de données **hiérarchiques**
- 1968 : Premier système de gestion de bases de données (**SGBDR**) dans un système d'exploitation
- 1970 : Thèse de Edgar.F.Codd à l'origine des **bases de données relationnelles**
- 1980 : Apparition des bases de données orientées objet
- 2005 : Déploiement des bases de **données non-relationnelles**

Historique et types de base de données

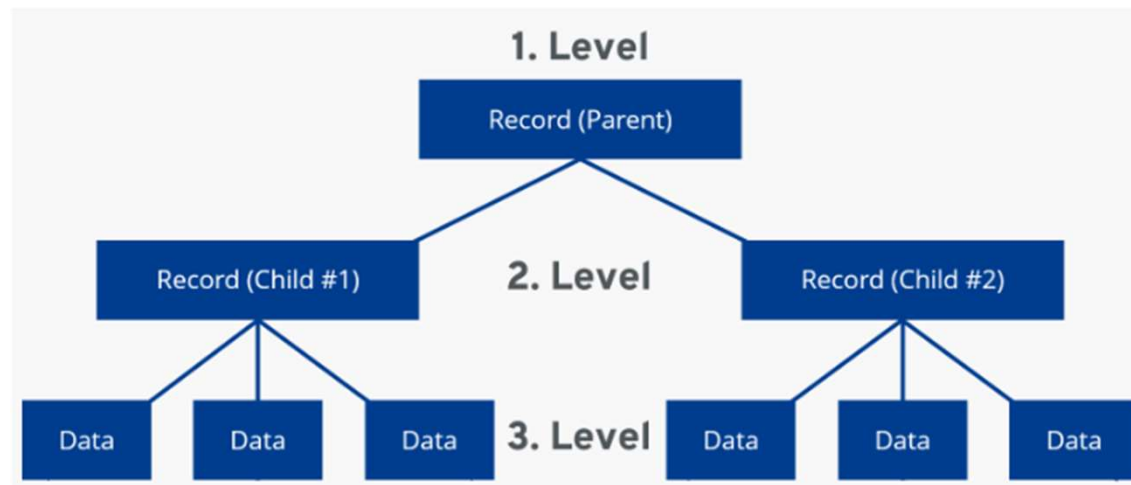


Années 60	Le terme « base de données » est apparu au début des années 60, lorsque les disques ont remplacé le stockage sur bande. Les premières bases de données étaient non relationnelles, c'est-à-dire qu'il s'agissait simplement de listes chaînées d'enregistrements sous forme libre. Les choses ont évolué dans la décennie suivante.
Années 70	Les années 70 ont vu le lancement des bases de données relationnelles, qui ont fini par devenir la norme. Contrairement aux bases de données précédentes, celles-ci donnaient accès à des tables normalisées, liées et interrogeables.
Années 80	L'informatique de bureau s'est largement démocratisée dans les années 80, tout comme les logiciels d'entreprise conviviaux interagissant avec des bases de données sous-jacentes.
Années 90	Dans les années 90, la programmation orientée objet (OOP) a rendu possible l'organisation des données par classe et attribut plutôt que par tableau et par champ, qui était plus sommaire.
Années 2000	Les bases de données non relationnelles ont fait leur retour dans les années 2000 sous la forme de bases de données NoSQL (pas seulement SQL). Simples et très évolutives, elles répondent aux exigences du Big Data et des applications Web en temps réel.

Types de base de données

1. Base de données hiérarchique

- Les bases de données **hiérarchiques** comptent parmi les plus anciennes bases de données.
- Ces bases de données permettaient de structurer l'information de façon hiérarchique : **chaque enregistrement dépendait d'un seul enregistrement**. Présenté sous forme d'arbre avec ses ramifications, cette façon de faire a très bien fonctionné pour certains projets.
- Mais rapidement, les **contraintes trop fortes** de dépendance (un seul enregistrement parent) ont amené au deuxième type de base de données.

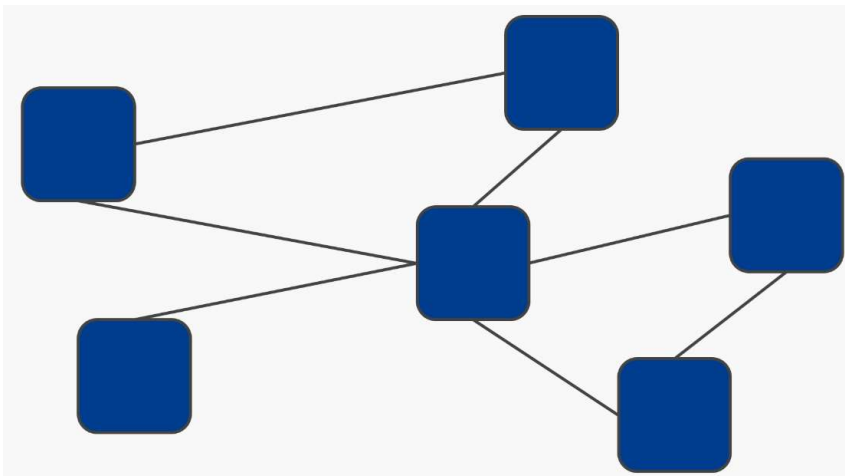


Types de base de données

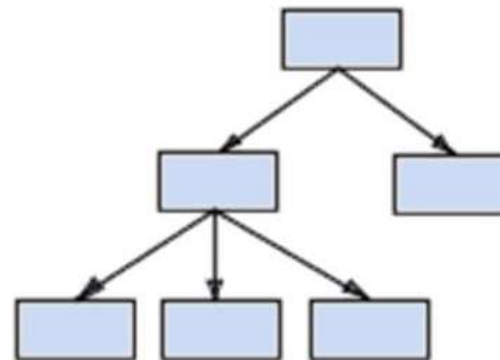
2. Les bases de données réseau

- Comment dire qu'un objet peut avoir plusieurs objets parents et plusieurs objets enfants ? Là où les bases de données hiérarchiques déclarent forfait, les bases de données réseau prennent le **relais** de façon très satisfaisante. En permettant les **relations n-n** (plusieurs parents / plusieurs enfants), les bases de données font un vrai bond en avant et permettent de **mimer plus fidèlement le monde réel**. D'une **structure en arbre**, les bases de données deviennent des **graphes**.
- Ce type de base de données (qui valu le **prix Turing** à son inventeur [C.W. Bachman](#)) apporte une **limitation non négligeable** : une trop grande dépendance entre les données et les programmes. Ce défaut empêche la diffusion de ce type de base de données au plus grand nombre et introduit son remplaçant.

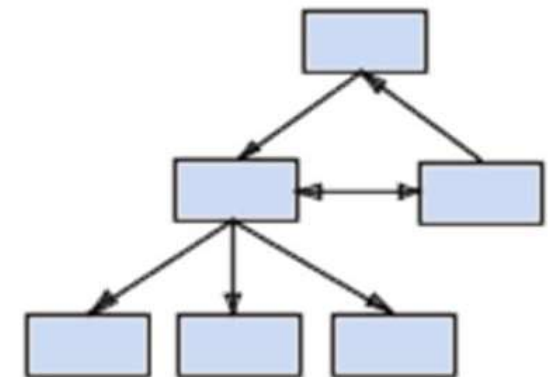
bases de données réseau



Modèle hiérarchique



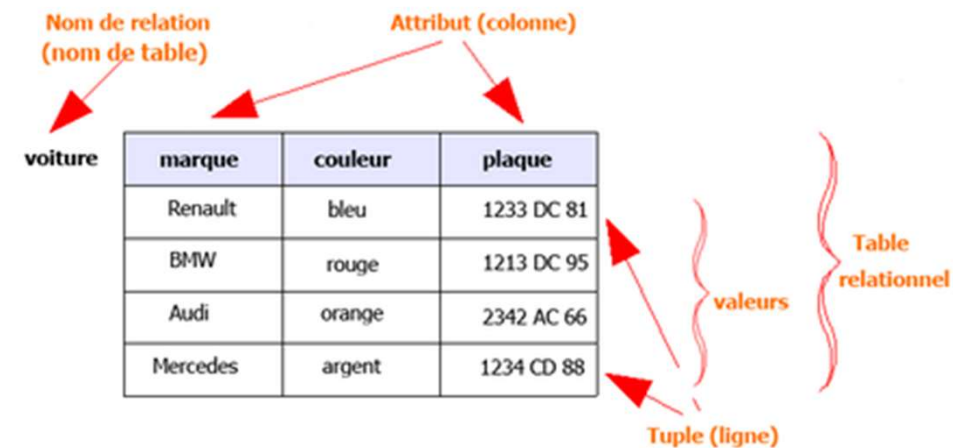
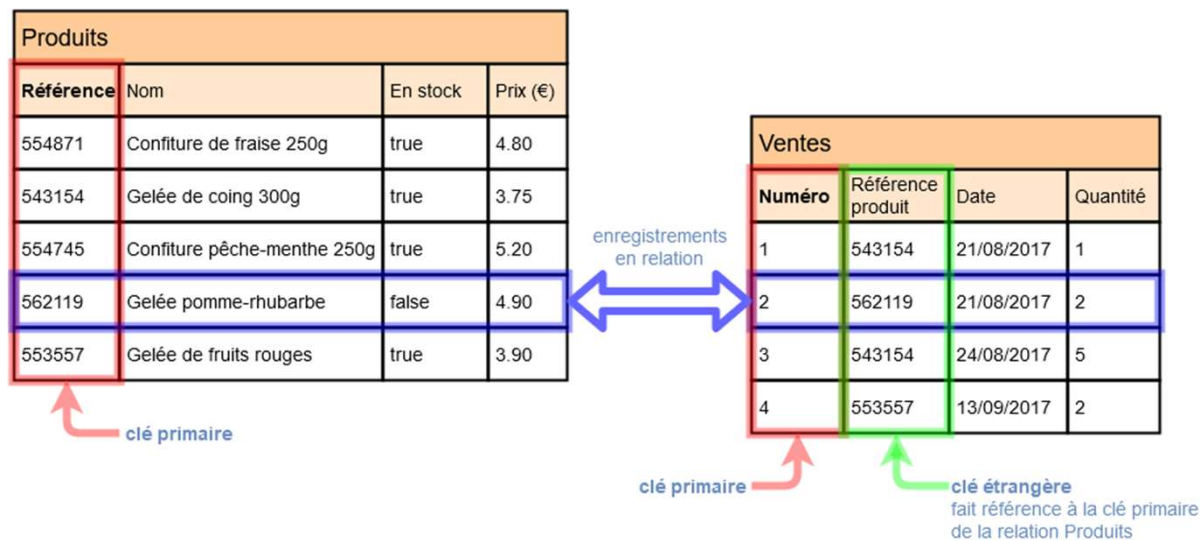
Modèle réseau



Types de base de données

3. Les bases de données relationnelles

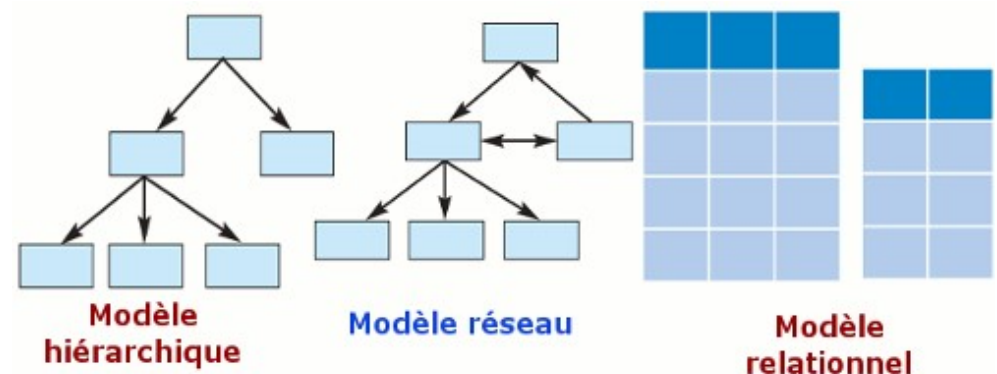
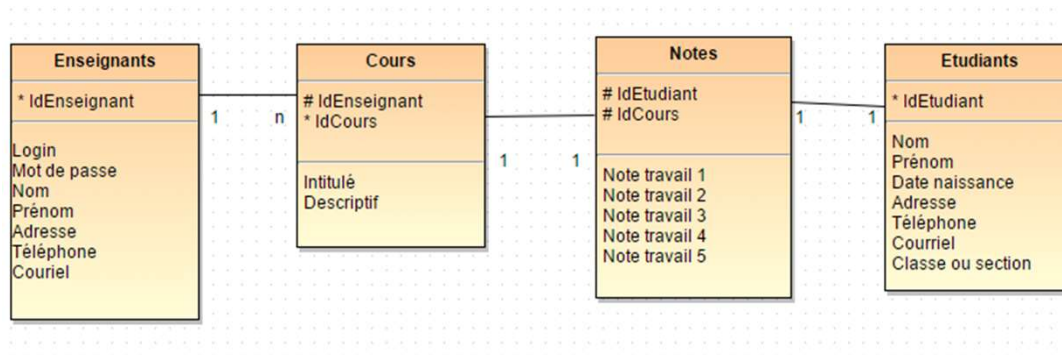
- C'est le type de bases que l'on connaît et que l'on pratique aujourd'hui. Basé sur l'[algèbre relationnel](#) et les [travaux de E.F. Codd](#), il permet de **modéliser** facilement et **sans grosse contraintes** les systèmes du **monde réel** et de créer des bases de données **simples à maintenir**, à faire **évoluer** et **indépendantes** de leur support.
- Les bases de données relationnelles sont constituées d'un ensemble de tableaux (**tables**). Au sein de ces tableaux, les **données sont classées par catégorie**. Chaque tableau comporte au moins une **colonne** correspondant à une **catégorie**. Chaque colonne comporte un certain nombre de données correspondant à cette catégorie.



Types de base de données

4. Les bases de données relationnelles ou Base de données SQL

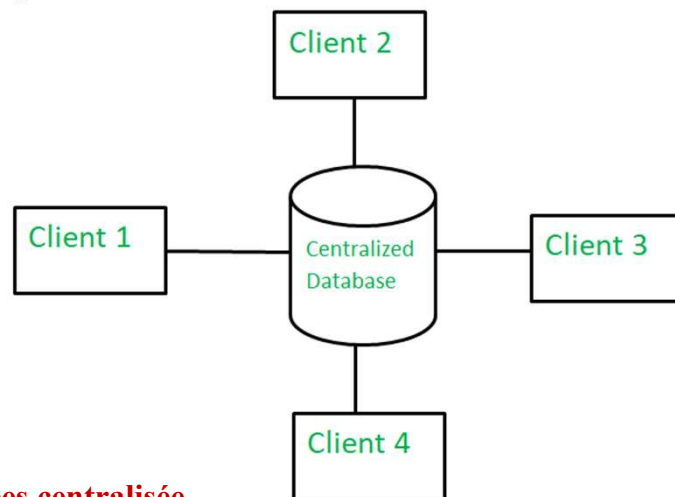
- L'API standard pour les bases de données relationnelles est le **Structured Query Language (SQL)**. Les bases de données relationnelles sont **facilement extensibles**, et de nouvelles catégories de données peuvent **être ajoutées** après la création de la database originale sans avoir **besoin de modifier** toutes les applications existantes.
- C'est la technologie majeure en bases de données depuis les **années 1980**. On peut cependant **leur reprocher** une nécessité de tout structurer qui ne convient pas pour représenter les **données mouvantes** et **hétérogènes** qu'on rencontre par exemple avec Internet.



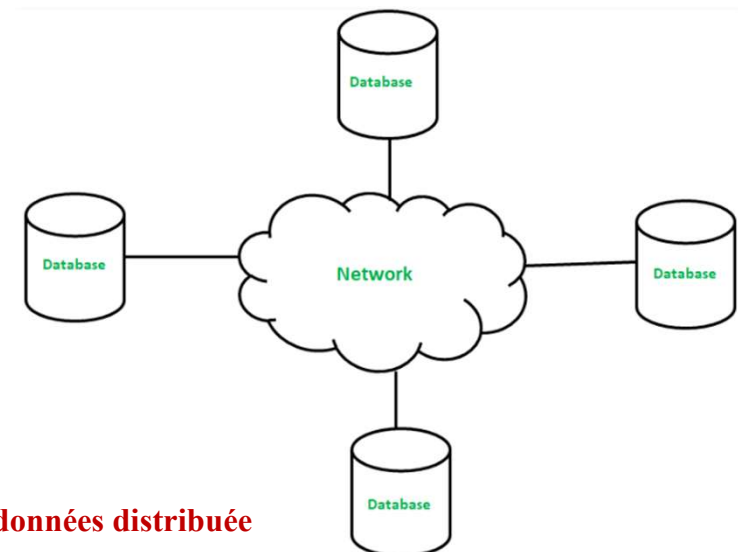
Types de base de données

5. Base de données distribuée

- Une BDD distribuée est une database dont **certaines portions sont stockées à plusieurs endroits physiques**. Le traitement est réparti ou répliqué entre différents points d'un réseau.
- Les bases de données distribuées **peuvent être homogènes ou hétérogènes**. Dans le cas d'un système de base de données **distribuée homogène**, tous les emplacements physiques fonctionnent avec le même hardware et tournent sous le même système d'exploitation et les mêmes applications de bases de données.
- Au contraire, dans le cas d'une database **distribuée hétérogène**, le hardware, les systèmes d'exploitation et les applications de bases de données peuvent varier entre les différents endroits physiques.



Base de données centralisée



Base de données distribuée

Types de base de données

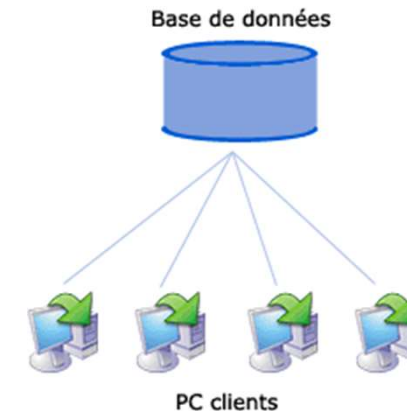
Base de données centralisée

Avantages :

- Étant donné que toutes les données sont stockées à un seul endroit, il est donc plus facile d'accéder aux données et de les coordonner.
- La base de données centralisée a une redondance de données très minimale puisque toutes les données sont stockées en un seul endroit.
- C'est moins cher par rapport à toutes les autres bases de données disponibles.

Désavantages:

- Le trafic de données dans le cas d'une base de données centralisée est plus.
- Si une défaillance du système se produit dans le système centralisé, toutes les données seront détruites.



Base de données distribuée

Avantages :

- Cette base de données peut être facilement étendue car les données sont déjà réparties sur différents emplacements physiques.
- La base de données distribuée est facilement accessible depuis différents réseaux.
- Cette base de données est plus sécurisée par rapport à une base de données centralisée.

Désavantages:

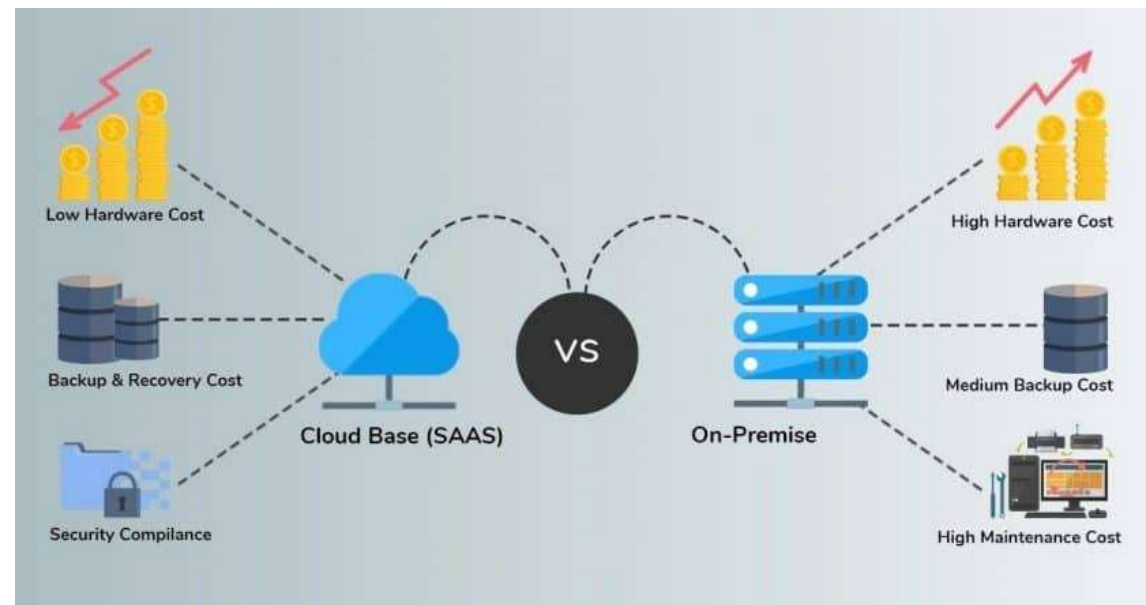
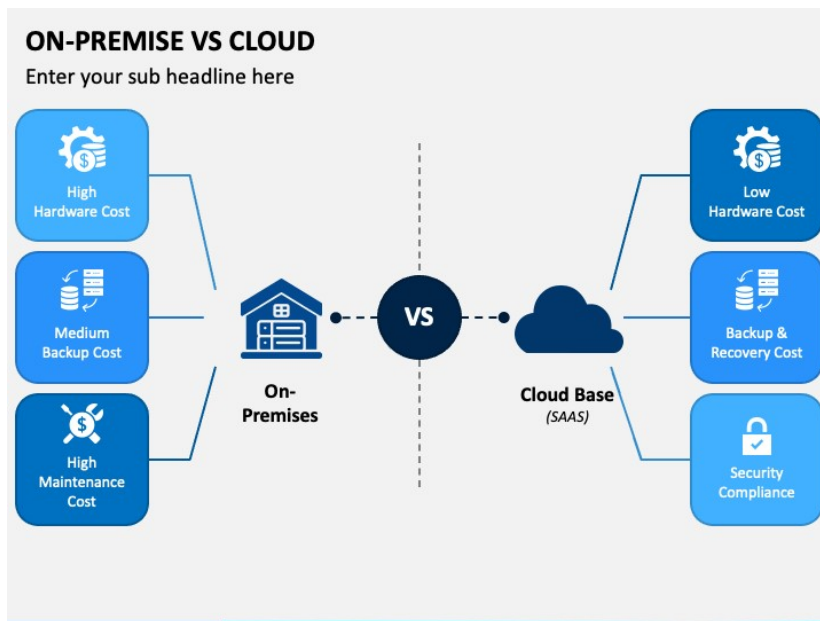
- Cette base de données est très coûteuse et difficile à maintenir en raison de sa complexité.
- Dans cette base de données, il est difficile de fournir une vue uniforme aux utilisateurs car elle est répartie sur différents emplacements physiques.



Types de base de données

6. Base de données cloud

- Dans ce cadre, elle est **optimisée ou directement créée pour les environnements virtualisés**. Il peut s'agir d'un cloud privé, d'un cloud public ou d'un cloud hybride.
- Les **bases de données cloud offrent plusieurs avantages** comme la possibilité de **payer pour la capacité de stockage** et la **bande passante** en fonction de l'usage. Par ailleurs, il est possible de **changer l'échelle sur demande**. Ces bases de données offrent aussi une **disponibilité plus élevée**.

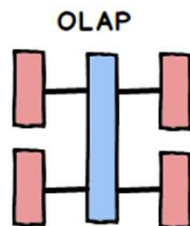
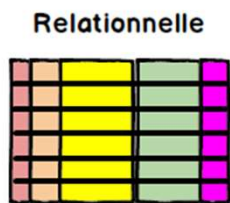


Types de base de données

7. Base de données NoSQL

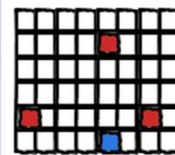
- Une base de données NoSQL est une **base de données « non relationnelle »**. Il est possible d'y stocker des données sous une forme **non structurée**, sans suivre de schéma fixe. Les jointures ne sont plus nécessaires, et le **scaling** est facilité.
- On utilise notamment les bases de données NoSQL pour les Data Stores distribués aux besoins élevés en capacité de stockage. Ainsi, **NoSQL est utilisé pour le Big Data** et les applications **web en temps réel**. Les géants de la technologie comme Twitter, Facebook ou Google collectent chaque jour plusieurs terabytes de données sur leurs utilisateurs.
- Le terme « **NoSQL** » signifie en fait « **Not Only SQL** » (pas seulement SQL). En effet, les bases de données relationnelles utilisent la syntaxe SQL pour le stockage et l'analyse de données.
- Ce n'est pas le cas d'une database non-relationnelle. Les systèmes NoSQL sont **compatibles avec une large variété de technologies** permettant le stockage de données structurées, non structurées, semi-structurées ou polymorphique.

Base de données SQL

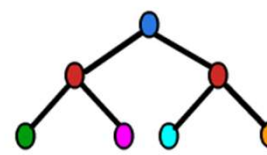


Base de données NoSQL

Orienté colonne



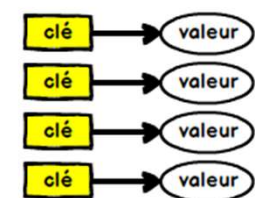
Document



Graphe



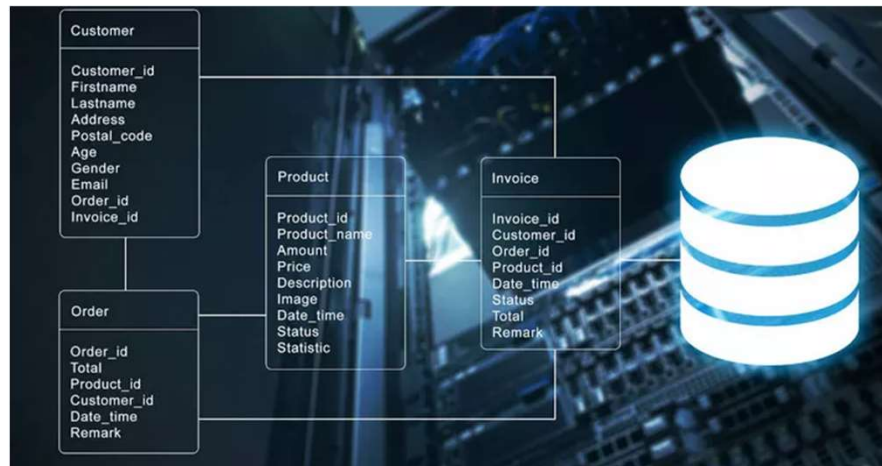
Clé-Valeur



Types de base de données

8. Base de données orientée objets

- Les **objets** créés à l'aide de langage de programmation orientés objets sont généralement stockés sur des bases de données relationnelles. Toutefois, en réalité, les **bases de données orientées objets** sont plus adaptées pour stocker ce type de contenu.
- Plutôt que d'être organisée **autour d'actions**, les bases de données orientées objets sont **organisées autour d'objets**. De même, au lieu d'être organisées autour d'une logique, elles sont organisées autour des données. Par exemple, un enregistrement **multimédia** au sein d'une **BDD relationnelle** peut être défini comme un **objet de données** plutôt que comme une valeur alphanumérique.

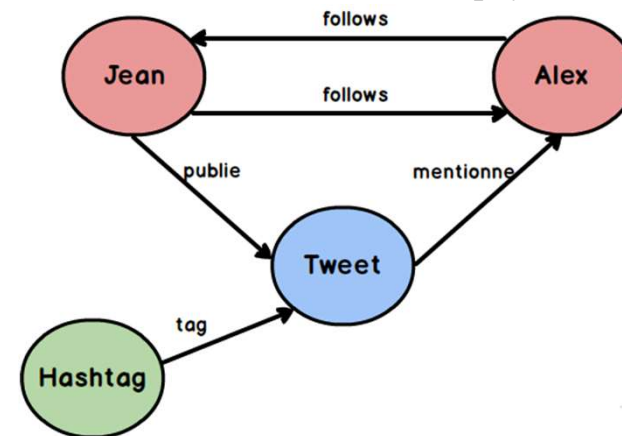
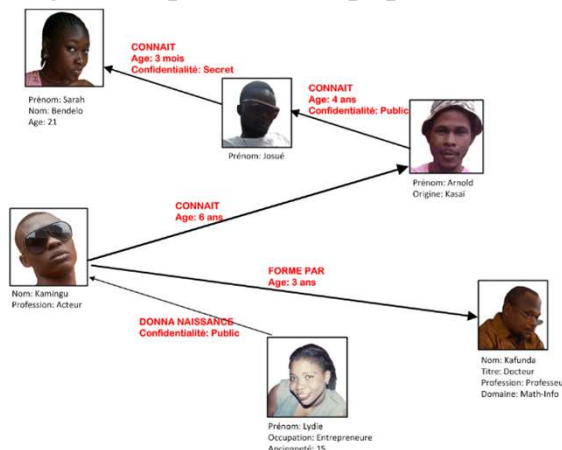


Avantages	Inconvénients
Les ensembles de données complexes s'enregistrent et s'interrogent rapidement et simplement.	Les bases de données orientées objet sont peu répandues.
Les identifiants des objets sont attribués automatiquement.	Dans certaines situations, la grande complexité peut engendrer des problèmes de performance.
Bonne compatibilité avec les langages de programmation orientés objet.	

Types de base de données

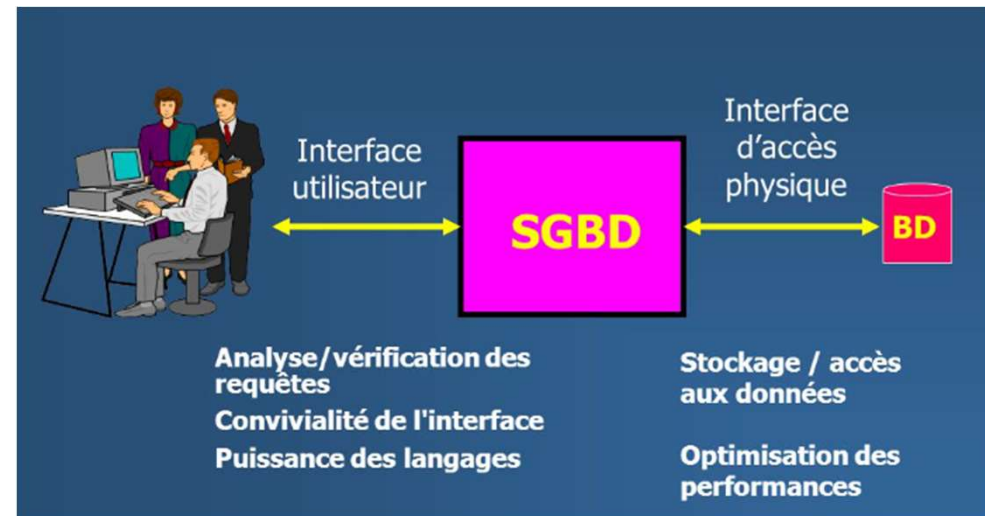
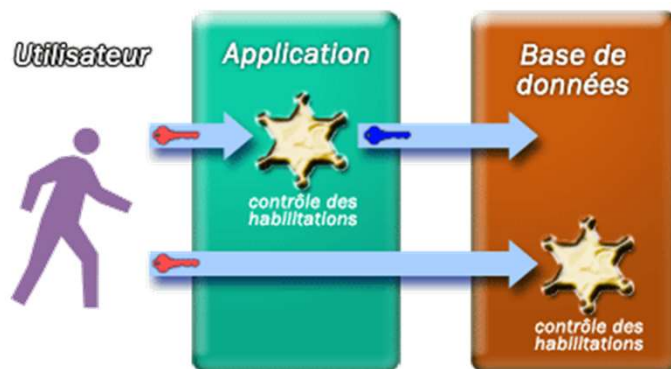
9. Base de données orientée graph

- Une base de données orientée graphe, ou graphe, est un **type de database NoSQL** utilisant la **théorie des graphes** pour stocker, cartographier et effectuer des requêtes sur les relations entre les données. Les **bases de données graphe** sont constituées de **noeuds** et de **bords**.
- Chaque **noeud** représente une **entité**, et chaque **bord** représente une **connexion** entre les noeuds. Les bases de données graphes gagnent en popularité dans le domaine des **analyses d'interconnexions**. Par exemple, les entreprises peuvent utiliser une **BDD graphe** pour **miner des données sur ses clients à partir des réseaux sociaux**.
- De plus en plus souvent, des bases de données jadis séparées sont combinées électroniquement sous forme de collections plus larges **que l'on** appelle les **Data Warehouses**. Les entreprises et les gouvernements utilisent ensuite des logiciels de **Data Mining** pour analyser les différents aspects des données. Par exemple, une agence gouvernementale peut procéder ainsi pour **enquêter sur une entreprise** ou une personne qui ont acheté une grande quantité d'équipement, même si les achats sont disséminés dans tout le pays ou répartis entre plusieurs subsidiaires.



Qu'est-ce qu'un DBMS ou SGBD ?

- Un DBMS est un **système de gestion de base de données**. Il s'agit d'un ensemble de programmes permettant aux utilisateurs d'**accéder** aux bases de données, de **manipuler** les données, de **générer** des rapports et de représenter les données. Ce système aide aussi à **contrôler** l'accès à la base de données.
- Il s'agit de **l'interface** entre la base de données et les utilisateurs finaux ou les programmes. Il permet aussi le **suivi** des performances, le **backup**, la **restauration**, ou la **configuration** des bases de données. Parmi les exemples les plus populaires, on peut citer **MySQL**, **Microsoft Access**, **Microsoft SQL Server**, **Oracle Database**, **FileMaker Pro** ou **dBase**.

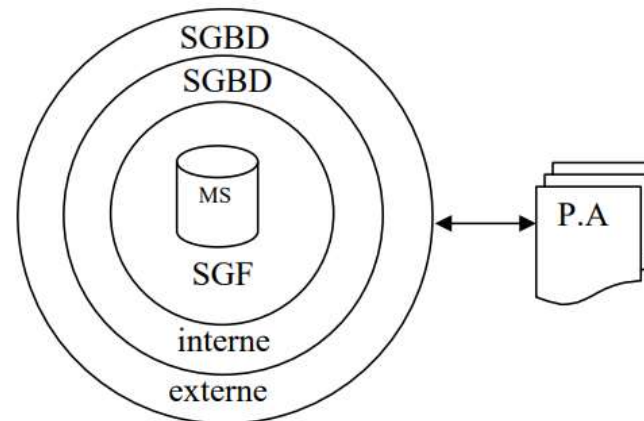


Qu'est-ce qu'un DBMS ou SGBD ?

Un SGBD se compose de **trois couches successives de fonctions**, empilées depuis les mémoires secondaires vers les usagers :

- 1) **Système de gestion de fichier (SGF)** : permet de stocker les données sur les mémoires secondaires (disques durs).
- 2) **SGBD interne** : permet la gestion des liens entre les données stockées dans les fichiers. Qui devient invisible aux programmes d'applications.
- 3) **SGBD externe** : assure l'analyse et l'interprétation des requêtes des usagers, et la mise en forme des données échangées avec le monde extérieur.

La figure ci-après illustre ces trois couches de fonctions qui constituent un SGBD.



les trois couches de fonctions qui constituent un SGBD.
P.A :programme d'application
M.S :mémoires secondaires

Objectifs d'un SGBD



Les objectifs principaux des systèmes de gestion de base de données sont résumés dans les points suivants :

- a) Indépendance physique
- b) Indépendance logique
- c) Manipulation des données par des non-informaticiens
- d) Efficacité des accès aux données
- e) Administration cohérente des données
- f) Non redondance des données
- g) Cohérence des données
- h) Partageabilité des données
- i) Sécurité des données

a. Indépendance physique :

- C'est un objectif essentiel des **SGBD**. Il s'agit de permettre de réaliser **l'indépendance des structures de stockage** aux **structures de données du monde réel**.
- Cela consiste à pouvoir **définir l'assemblage** des données élémentaires entre elles dans le système informatique **indépendamment** de l'assemblage **réalisé** dans le monde réel, en tenant compte seulement des **critères de performances** et de **flexibilité d'accès**.
- Ce qui permettrait de **'transporter'** plus ou moins facilement des logiciels et des données entre des **équipements hétérogènes**.

Objectifs d'un SGBD



b. Indépendance logique :

- Il s'agit de permettre à chaque groupe de travail réalisant une application de pouvoir **assembler différemment** les données pour former des entités et des associations.
- De plus, chacun doit pouvoir se **concentrer** sur les éléments constituant son centre d'intérêt, c'est à dire que chacun doit pouvoir **ne connaître et ne voir** qu'une partie des données.

c. Manipulation des données par des non-informaticiens :

- Si les objectifs **a** et **b** sont atteints, les utilisateurs **non-informaticiens** verront les données indépendamment de leur implantation en machine, et peuvent alors les manipuler, les interroger et les mettre à jour, au moyen des langages non procéduraux, c'est à dire en **décrivant les données** qu'ils souhaitent retrouver **sans décrire la manière** de les retrouver qui est propre à la machine.
- Par exemple, vous arrivez dans **une ville inconnue** et vous prenez un taxi pour vous rendre chez X, c'est le chauffeur de taxi qui, pour répondre à votre requête, prendra le chemin le plus court pour accéder chez X.

Objectifs d'un SGBD



d. Efficacité des accès aux données :

- Il s'agit d'abord de **fournir** des **langages de manipulation** de données très **efficaces** permettant d'accéder **rapidement** aux données.
- D'autre part, pour les **non-informaticiens**, il doit être possible d'utiliser un **langage non procédural** dont l'implantation doit être **très efficace**, notamment au niveau des **accès aux disques** qui restent le goulot d'étranglement essentiel des **SGBD**.

e. Administration cohérente des données :

- **L'administration des données** est l'ensemble des fonctions privilégiées ayant trait à : la **définition des structures** de stockage, la **création des espaces de travail** pour les utilisateurs, l'**attribution** aux utilisateurs des **droits d'accès**, ...etc.
- **L'extrême importance** de ces fonctions, rend **obligatoire** la mise sous contrôle du **SGBD** par un ou plusieurs **personnes hautement qualifiées** (appelées **administrateurs du système DBA**).
- L'administration des données est ainsi souvent **centralisée**. L'objectif est seulement de permettre une administration **cohérente** et **efficace** des données.

Objectifs d'un SGBD



f. Non redondance des données :

- Dans les systèmes classiques à fichiers non intégrés, **chaque application** possède des **données propres**. Ceci conduit généralement à de nombreuses **duplications de données** avec, outre la **perte de place** en **mémoire** secondaire associée (disque dur), un **gâchis** (gaspillage) important en moyens humains pour saisir et maintenir à jour **plusieurs fois** les mêmes données.
- Avec une approche '**base de données**', les fichiers plus ou moins **redondants** sont intégrés en un **seul fichier partagé** par les diverses applications.

g. Cohérence des données

- Les **données** peuvent être soumises à **certaines règles**. Par exemple, une quantité commandée Q d'un produit P ne doit pas être supérieurs à la quantité S en stock du même produit.
- Le SGBD doit donc veiller à ce que les applications **respectent ces règles** lors des **modifications** des données et doit ainsi assurer la **cohérence des données**.

Objectifs d'un SGBD



h. Partageabilité des données :

- Cet objectif consiste à permettre aux applications de **partager les données** de la base dans le temps mais aussi **simultanément**.
- Une application doit pouvoir accéder aux données comme si elle **était seule à les utiliser**, sans attendre mais aussi sans savoir qu'une autre application peut les **modifier en même temps**.

g. Sécurité des données :

- Les données doivent **être protégées** contre les accès **non autorisés** ou **mal intentionnés**. Il doit exister des mécanismes pour **autoriser, contrôler** ou **enlever les droits d'accès** de n'importe quel usager à un ensemble de données.
- Par exemple, un employé pourra connaître les salaires des personnes qu'il dirige, mais pas des autres employés de l'entreprise.



Quelles sont les meilleures bases de données SQL à l'heure actuelle



- ❑ Le choix d'une bonne base de données est très important pour votre entreprise. Il faut qu'elle soit **facile à utiliser**, **sécurisée**, avec un **bon suivi de développement** peut permettre **d'augmenter la productivité**. Il est donc indispensable de bien étudier les **avantages et les inconvénients de chaque d'entre elle**. La database que vous choisirez doit **s'adapter à l'écosystème** de votre entreprise.
- ❑ Plusieurs **questions essentielles doivent être posées** : quelle quantité d'éléments devrez-vous gérer, quel est le temps de réaction attendu par vos clients, combien de clients avez-vous, comment s'adaptera-t-elle si votre nombre de clients et de transactions augmente, comment vous la surveillerez pour éviter les down times, avez-vous besoin d'une database relationnelle ou NoSQL, et comment se comportera-t-elle en cas de plantage ou de problème ?
- ❑ Actuellement, le **marché est dominé par DB2, SQL Server, Oracle et IBM**. Sur **Windows**, SQL Server est généralement la BDD de prédilection, tandis que **Oracle et DB2** sont les plus populaires sur les écosystèmes **Mainframe/Unix ou Linux**. Pour vous aider celle qui vous convient, voici notre sélection des meilleures bases de données.



TOP 5 SERVEURS DE BASE DE DONNEES



Quelles sont les meilleures bases de données SQL à l'heure actuelle ?

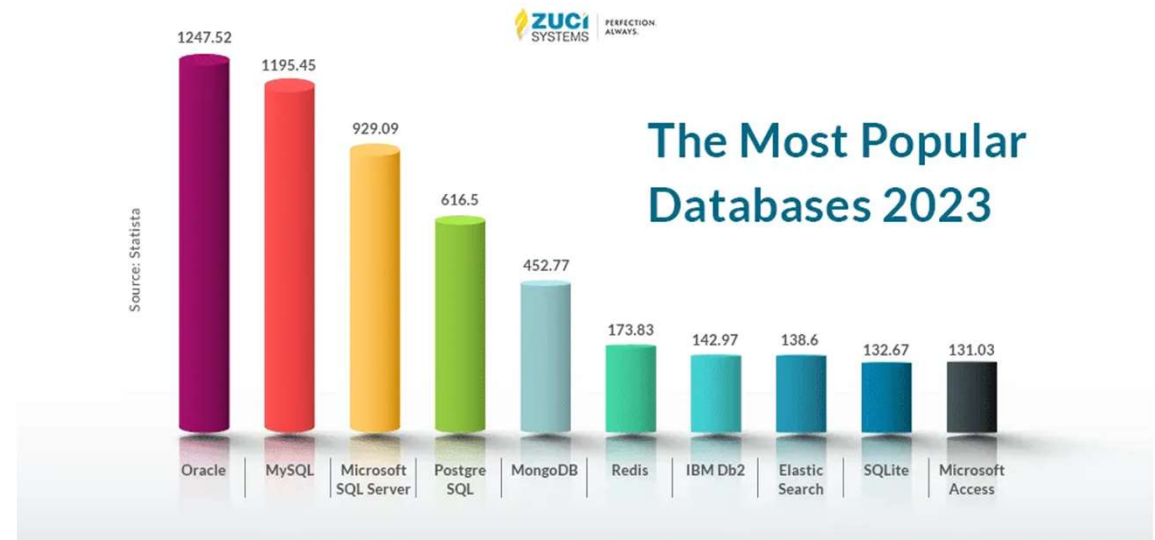


Microsoft SQL Server, la base de données Windows

- Développée par Microsoft, **SQL Server**, elle est **compatible exclusivement avec Windows**. Cette database est simple à maîtriser, et de nombreuses personnes y sont formées. L'intégration avec **Microsoft Azure** a permis d'augmenter sa flexibilité et ses performances.
- De plus, le **cloud** permet désormais d'administrer des informations en provenance d'autres serveurs, ce qui la rend plus utile.

Oracle, la base de données la plus populaire pour Linux/Unix

- La database **Oracle** peut fonctionner sur presque tous les systèmes. Elle est très populaire, et de **nombreuses personnes sont formées à la maîtriser**.
- Par ailleurs, elle propose de nombreux **outils** orientés vers le **monitoring** et l'**administration**.



Quelles sont les meilleures bases de données SQL à l'heure actuelle ?



IBM DB2, la base de données Mainframe la plus populaire

- Après Oracle, **IBM DB2** est la deuxième la plus utilisée sur les écosystèmes **Unix/Linux**. Pour **Mainframe**, il s'agit du choix le plus populaire.
- Là encore, de nombreuses personnes sont formées à l'utiliser, même si elle compte moins d'adeptes qu'**Oracle**.

Teradata, la meilleure base de données pour le Big Data

- Teradata est spécialement conçue pour le Big Data. De fait ses **capacités de stockage** et d'**analyse de données** sont **colossales**. Pour une stratégie **Big Data**, il s'agit sans conteste de la meilleure option à votre disposition.

SAP Sybase, l'ancien leader du marché

- Par le passé, cette database était très populaire et dominait largement le marché. Aujourd'hui, elle n'est plus aussi utilisée, mais **reste une solution très pertinente** en termes de **scalabilité** et de **performances**.

Informix, une database rachetée par IBM

- Tout comme **SAP Sybase**, **Informix** a **perdu de sa superbe** à la fin des années 90. Suite à une série de mauvaises décisions en termes de management, elle a fini par être rachetée par IBM.
- Elle n'existe plus aujourd'hui sous sa forme initiale. Malgré tout, ses **fondations restent utilisées par certains outils et applications IBM**.

Quelles sont les meilleures bases de données NoSQL ?



Parmi les bases de **données NoSQL**, on dénombre de nombreuses sous-catégories. Chacune de ces catégories se distingue par des caractéristiques spécifiques. Voici les **quatre principaux groupes de bases de données NoSQL**, et les meilleures de chacune de ces catégories.

Les bases de données NoSQL orientées clé-valeur

- Ces bases de données sont idéales pour **accéder aux données par le biais d'une clé**. La spécificité est que les données peuvent être stockées **sans définir de schéma spécifique**. Ces bases de données sont **très efficaces** pour la lecture et l'écriture, et conçues pour **s'adapter massivement** et proposer un **temps de réaction extrêmement rapide**.
- Les éléments sont généralement stockés au sein de structures complexes comme le **BLOB**. Les références dans cette catégorie de bases de données sont **Redis, Riak, Oracle NoSQL et Microsoft Azure Table Storage**. **Redis** est une base gratuite et open source, **Riak** est entièrement dédiée aux clés-valeur et idéale pour le stockage de document et les fonctionnalités de recherche.

Clé	Identité		
	Suffixe	Prénom	Nom
1		Mick	Jameson
2	Dr	Jack	Mickeal
3	PhD	Min	Lee

Clé	Identité		
	Email	Téléphone	Adresse
1	mick.jam@gmail.com	+33XXXXXX	
2	jack.mich@outlook.com	+33XXXXXX	75000 Paris
3	min.lee@orange.com	+33XXXXXX	

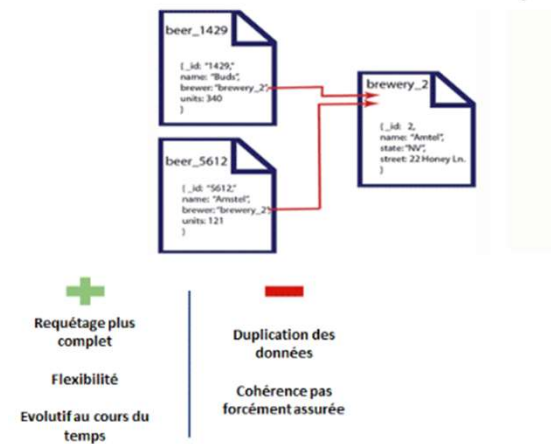
Clé	Valeur
1	https://adresseweb.com
2	356
3	mail: monmail@gmail.com date: 25/10/2020 13:42:12

Quelles sont les meilleures bases de données NoSQL ?

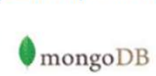


Les bases de données NoSQL orientées document

- Cette catégorie de bases de données **repose sur divers formats (JSON, XML)** et offre la **capacité de changer** de schéma sans avoir à **arrêter la database**. Les développeurs peuvent télécharger des documents indexés et y accéder par le biais de moteur de stockage de la base de données. La **flexibilité** de ces bases de données les rend très **polyvalentes**.
- Parmi les meilleures bases de données NoSQL orientées document, on peut citer **Mongo DB** et **Couchbase Server**. **Mongo DB** est l'une des bases de données les plus populaires à l'heure actuelle, toutes catégories confondues. Elle permet de prendre en charge aussi bien des **données structurées** que des **données non structurées**, et ses **performances** et sa **scalabilité** sont **excellentes**. De nombreuses personnes sont formées pour la maîtriser.
- **Couchbase Server** est une database **Open Source** licenciée sous Apache. Son avantage principal est sa **console d'administration très intuitive** permettant d'accéder facilement à de grandes quantités de données. En revanche, elle ne **permet pas de garantir** l'intégralité des données à 100%.
- Parmi les meilleures bases de données NoSQL orientées document, **on peut aussi citer Mark Logic Server**. Son intégrité des données et sa compatibilité XML, JSON et RDF en font une référence. **Mark Logic Server** est compatible avec les OS Windows, Solaris, Red Hat, Suse, CentOS, Amazon Linux et Mac OS. On peut enfin citer **Elastic Sarche, Ravendb, Apache Jena et Pivotal GemFire**.



Exemples d'implémentation



Quelles sont les meilleures bases de données NoSQL ?



Les bases de données NoSQL orientées colonne

- Les bases de données NoSQL orientées colonne **représentent la valeur des données sous forme de colonne**, ce qui permet à l'utilisateur de cartographier les clés et les valeurs et de les regrouper en **structures**. Ces bases de données sont principalement utilisées dans les environnements où il est nécessaire d'accéder à de nombreuses colonnes elles-mêmes dotées de nombreuses rangées. Elles sont surtout utiles pour le **traitement** et **l'analyse d'événements**, la **gestion de contenu** et **l'analyse de données**.
- Parmi les meilleures bases de données NoSQL orientées colonne, on peut citer **Apache Cassandra**, un **moteur créé par Facebook** et désormais distribué gratuitement. Cassandra est recommandé pour les bases de données regroupant d'immenses quantités de données.
- Précisons qu'une **version entreprise** intitulée **Datastax Enterprise** est également disponible. **Cassandra** est compatible ASCII, bigint, BLOB, Boolean, counter, decimal, double, float, int, text, timestamp, UUID, VARCHAR et variant. L'autre référence dans cette catégorie est **Apache Hbase**, conçue pour prendre en charge de multiples accès lecture et écriture en temps réel à d'immenses quantités de données.

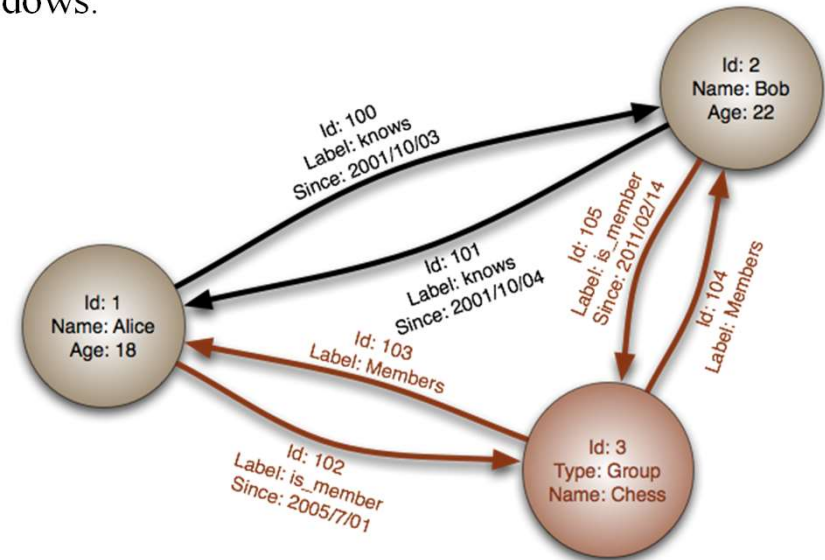
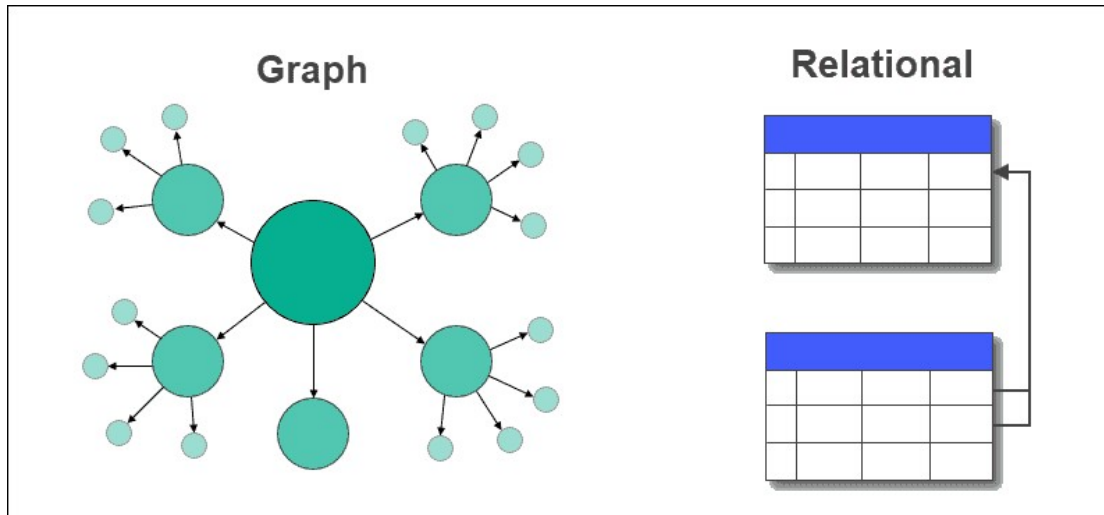
Etudiants				
CLE	Nom	Prénom	Age	Taille
	Moulin	Mathieu	16	180
CLE	Nom	Prénom	Age	
	Dupond	Jean	23	
CLE	Nom	Prénom	Age	Taille
	Martin	Simon	22	175

Quelles sont les meilleures bases de données NoSQL ?



Les bases de données NoSQL orientées graphe

- Les bases de données **NoSQL orientées graphe** sont **focalisées sur les propriétés et les relations** qui les unissent. Elles utilisent la **théorie des graphes** pour connecter les bases de données. Chaque élément est connecté à son élément adjacent. Ces bases de données sont recommandées si vos données sont **interrelationnelles**, comme sur les **réseaux sociaux**, dans la **détection de fraude** ou encore les mises à jour en temps réel.
- Les références dans cette catégorie sont **Neo4J** et **Infinite graph**. **Neo4j** prend en charge **l'intégration** des données, offre une grande **disponibilité**, et le **clustered scaling**. Son panel d'administration est également très bon. **Infinite Graph** est une base de données licence-only compatible avec macOS, Linux et Windows.



Transactions de bases de données et Propriétés ACID

- Au sein d'une base de données, le terme de « **Transaction** » désigne **les opérations apportant des modifications aux données**. Par exemple, un virement bancaire provoquant le débit du compte de l'émetteur et le crédit du compte du bénéficiaire est une **transaction**.
- Autrement, le terme de **transaction** désigne **n'importe quelle opération** effectuée au sein d'une base de données. Il peut s'agir par exemple de la création d'un **nouvel enregistrement** ou d'une **mise à jour** des données.
- Or, le moindre changement apporté à une base de données doit être effectué avec une extrême rigueur. Dans le cas contraire, **les données risquent d'être corrompues**.
- Ces transactions doivent toutefois présenter **quatre propriétés** visant à garantir **leur validité** même en cas d'**erreur** ou de **pannes informatiques**. Ces quatre propriétés sont l'**atomicité**, la **cohérence**, l'**isolation** et la **durabilité**. Afin de mémoriser facilement ces attributs, *Andreas Reuter* et *Theo Härder* ont inventé l'acronyme » **ACID** » en 1983.
- En appliquant les propriétés **ACID** à **chaque modification** effectuée dans une base de données, il est **plus facile de maintenir son exactitude et sa fiabilité**.



Transactions de bases de données et Propriétés ACID

Atomicité :

- L'**atomicité** des transactions au sein des bases de données signifie que **tous les changements apportés aux données sont effectués totalement**, ou **pas du tout**. Soit tous les changements apportés par la transaction sont enregistrés pour la postérité, soit aucun ne l'est.
- En outre, l'atomicité permet **d'éviter que les changements prennent effet** en cas de panne de l'application ou du serveur de la base de données en plein milieu de la transaction. Ainsi, la base de données ne risque pas **d'être corrompue** par des opérations **imprévisibles** et à moitié **complétées**.



Cohérence :

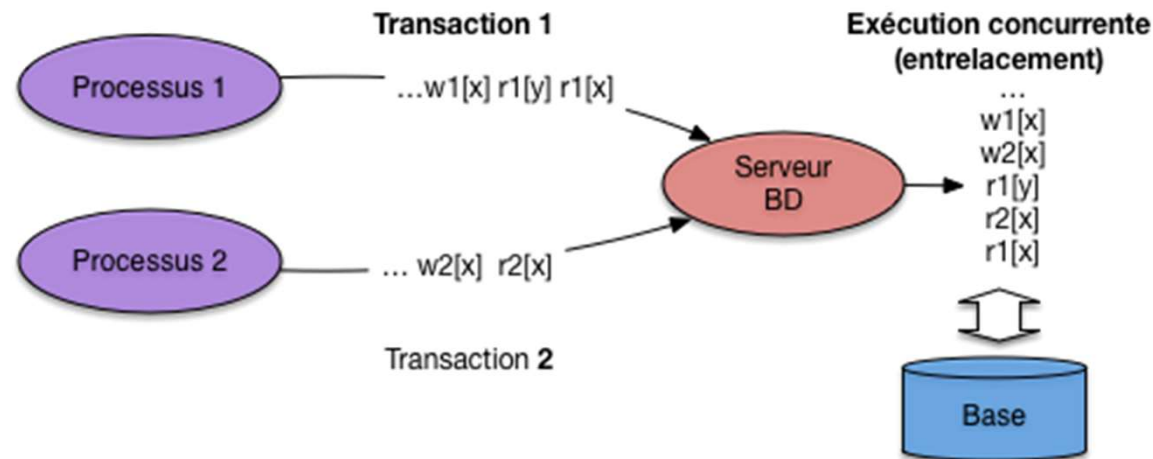
- La **cohérence** signifie que les transactions doivent **respecter les contraintes d'intégrité des données de la base de données**. Ainsi, une transaction qui commence avec un ensemble de données cohérent (respectant les contraintes d'intégrité) doit résulter par un ensemble de données cohérent.
- Pour maintenir la cohérence, [un système de base de données DBMS](#) peut **abandonner les transactions qui risquent de provoquer une incohérence**. Précisons que cette cohérence ne s'applique pas aux étapes intermédiaires de la transaction.
- Pour assurer la cohérence au fil du temps, **il est possible d'utiliser une base de données temporelle**. Ceci permet de spécifier les contraintes de données qui doivent traverser la dimension temporelle.



Transactions de bases de données et Propriétés ACID

Isolation :

- L'isolation signifie que **les écritures et lectures des transactions réussies ne seront pas affectées par les écritures et lectures d'autres transactions**, qu'elles **soient** ou **non réussies**. Les transactions isolées peuvent être « **sérialisées** » (**en série**) , ce qui signifie que l'état final du système peut être atteint en effectuant les **transactions une par une**.
- Pour décrire comment l'isolation peut être atteinte, on emploie souvent **les termes des transactions « optimistes » ou « pessimistes »** . Dans le cas des **transactions optimistes**, le logiciel de gestion des transactions considère de manière générale que les autres transactions sont réussies et qu'elles ne lisent ou n'écrivent pas deux fois au même endroit. Il permet aussi les lectures et les écritures aux données modifiées par d'autres transactions. Si une transaction est avortée ou si des conflits d'ordre surviennent, les deux transactions sont annulées et réinitialisées.



Transactions de bases de données et Propriétés ACID

- Dans le cas des **transactions pessimistes**, les ressources sont verrouillées jusqu'à la fin de la transaction pour empêcher toute **interférence**. Si une transaction nécessite des ressources utilisées par une autre, elle devra patienter. Bien souvent, cette attente est inutile et entame les performances.
- Il existe en outre de nombreuses **approches hybrides** combinant transactions optimistes et pessimistes. Ceci permet généralement de profiter des performances supérieures des transactions optimistes, et de la progression garantie des transactions pessimistes.



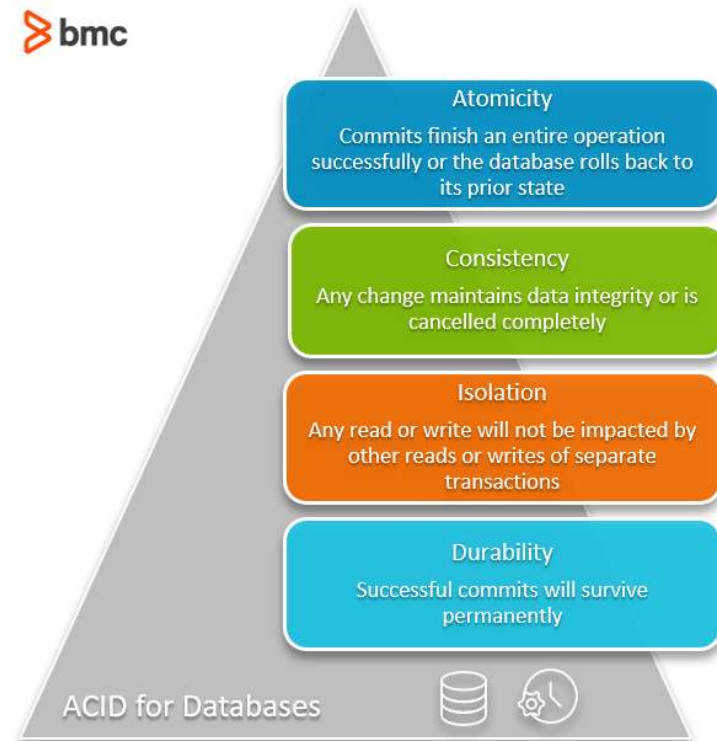
Durabilité :

- Enfin, la **durabilité** garantit que les transactions réussies survivront de façon permanente et ne seront pas affectées par d'éventuelles pannes ou problèmes techniques. Les **changements apportés** aux données doivent être **permanents**. Plus précisément, ce sont les effets logiques des données modifiées sur les futures transactions qui doivent être permanents.
- La simple écriture des données sur le disque ne suffit pas pour atteindre la durabilité. Pour cause, le disque peut par exemple tomber en panne. Il est **nécessaire que le DBMS écrive des logs** sur les changements effectués. Ces logs doivent être **permanents**, et éventuellement **redondants**.

Transactions de bases de données et Propriétés ACID

Durabilité (suite) :

- En cas de panne du disque, de l'OS ou du **DBMS**, la base de données sera redémarrée et le **DBMS pourra vérifier si les données sont synchronisées avec les logs**. En effet, les logs contiennent les effets de toutes les transactions effectuées. Ce n'est pas forcément le cas des fichiers de données. Si des informations sont **manquantes**, les **opérations enregistrées dans les logs** seront à nouveau effectuées.



SQL (Structured Query Language : langage de requêtes structurées



- Historiquement, c'est en **1970** que le **SQL, langage de requêtes structuré**, a été inventé par **E. F. Codd** (directeur de recherche du centre **IBM** de San José). Par la suite, de nombreux langages ont fait leur apparition tels que **IBM Sequel** (*Structured english query language*) en **1977**, IBM Sequel /2, IBM System/R et IBM DB2.
- Ce sont ces langages qui ont donné naissance au **standard SQL**, normalisé en **1986** par l'ANSI pour donner SQL/86. Puis en 1989 la version SQL/89 a été approuvée. La norme SQL/92 a désormais pour nom **SQL 2**.

Qu'est ce que le SQL?

- SQL** est un **langage de requête structuré**, qui est un langage informatique permettant de stocker, manipuler et récupérer des données stockées dans une base de données relationnelle.
- SQL** est la **langue standard** pour les systèmes de base de données relationnelle. Tous les systèmes de gestion de base de données relationnelle (SGBDR) tels que **MySQL, MS Access, Oracle, Sybase, Informix, Postgres et SQL Server** utilisent **SQL** comme langage de base de données standard.



SQL (Structured Query Language : langage de requêtes structurées



Pourquoi SQL ?

SQL est très populaire car il offre les avantages suivants:

- Permet aux utilisateurs **d'accéder aux données** des systèmes de gestion de base de données relationnelle.
- Permet aux utilisateurs de **décrire les données**.
- Permet aux utilisateurs de **définir les données** dans une base de données et de les **manipuler**.
- Permet **d'intégrer** dans d'autres langages en utilisant des **modules SQL**, des bibliothèques et des pré-compilateurs.
- Permet aux utilisateurs de **créer** et de **supprimer des bases** de données et des tables.
- Permet aux utilisateurs de **créer une vue**, une **procédure stockée** et des **fonctions** dans une base de données.
- Permet aux utilisateurs de **définir des autorisations** sur les tables, les procédures et les vues.

Processus SQL

- Le **traitement des requêtes** inclut des **traductions** sur des **requêtes de haut niveau** en **expressions de bas niveau** pouvant être utilisées au **niveau physique** du système de fichiers, une **optimisation** de la requête et une **exécution réelle** de la requête pour obtenir le **résultat réel**.

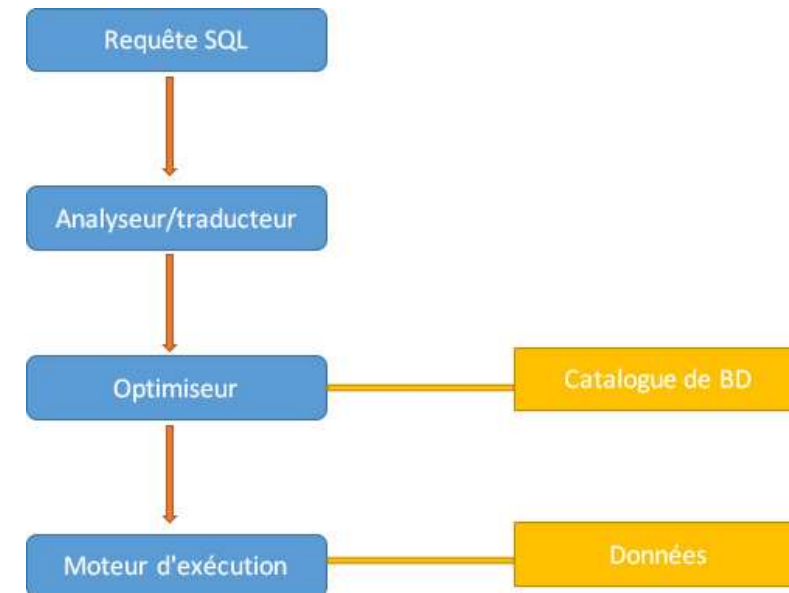
SQL (Structured Query Language : langage de requêtes structurées



Processus SQL

Etape 1 :

- **Analyseur** - lors de l'appel d'analyse, la base de données effectue les contrôles suivants: **contrôle de syntaxe**, **contrôle sémantique** et **contrôle de pool partagé**, après **conversion** de la requête en **algèbre relationnelle**.
- **Analyse dure et analyse douce** - S'il y a une nouvelle requête et que son code de hachage n'existe pas dans le pool partagé, cette requête doit passer par les étapes supplémentaires connues sous le nom **d'analyse dure**. Sinon, si le code de hachage existe, la requête ne passe pas par des étapes supplémentaires. . Il passe directement au moteur d'exécution (voir schéma). Ceci est appelé **analyse douce**. L'analyse dure comprend les étapes suivantes - **optimiseur** et **génération de sources de lignes**.



SQL (Structured Query Language : langage de requêtes structurées

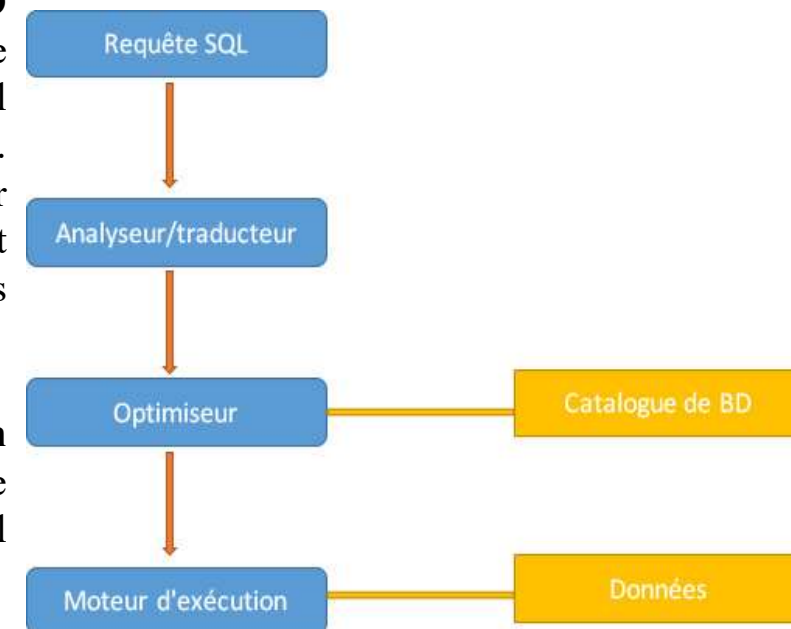


Processus SQL

Etape 2 :

❖ **Optimiseur** - au cours de la **phase d'optimisation**, la base de données doit effectuer une **analyse syntaxique stricte** au moins pour une instruction **LMD** unique et effectuer **une optimisation** au cours de cette analyse. Cette base de données n'optimise jamais la **LDD** à moins d'inclure un **composant LMD**, tel qu'une **sous-requête**, qui nécessite une optimisation. Il s'agit d'un processus dans lequel plusieurs **plans d'exécution de requêtes** pour satisfaire une requête sont examinés et le **plan de requêtes le plus efficace** est satisfait pour son exécution. Le **catalogue** de base de données stocke les plans d'exécution, puis l'optimiseur **transmet le plan d'exécution** le moins coûteux.

❖ **Génération de source en ligne** - Est un logiciel qui reçoit un **plan d'exécution optimal** de l'optimiseur et génère un **plan d'exécution itératif utilisable** par le reste de la base de données. le **plan itératif** est le **programme binaire** qui, lorsqu'il est exécuté par le moteur SQL, produit le résultat.



Etape 3 :

❖ **Moteur d'exécution** - Exécute enfin la requête et affiche le résultat souhaité.

SQL (Structured Query Language : langage de requêtes structurées



SQL est un triple langage :

- ❑ **Un langage de définition de données (LDD**, en anglais **DDL *Data Definition Language***), est utilisé pour **définir** la base de données, **créer** des tables, ainsi que d'en **modifier** ou en **supprimer** ; Les commandes DDL sont **validées automatiquement**, c'est-à-dire que les modifications effectuées par les commandes DDL sur la base de données sont enregistrées de **manière permanente**.
- ❑ **Un langage de manipulation de données (LMD**, en anglais **DML, *Data Manipulation Language***), ce qui signifie qu'il permet de **sélectionner**, **insérer**, **modifier** ou **supprimer** des **données** dans une table d'une base de données relationnelle. Les commandes LMD **ne sont pas validées automatiquement** et peuvent être **annulées**.
- ❑ **Un langage de contrôle de données (LCD**, en anglais **DCL, *Data Control Language***), Les commandes LCD sont utilisées pour **contrôler la visibilité des données** dans la base de données, comme la **révocation de l'autorisation** d'accès pour l'utilisation des données dans la base de données.

SQL (Structured Query Language : langage de requêtes structurées



SQL est un triple langage :

LDD - Langage de définition de données

Commande	Description
CREATE	Crée une nouvelle table, une vue d'une table ou un autre objet de la base de données.
ALTER	Modifie un objet de base de données existant, tel qu'une table.
DROP	Supprime une table entière, une vue d'une table ou d'autres objets de la base de données.

LMD - Langage de manipulation de données

Commande	Description
SELECT	Récupère certains enregistrements d'une ou de plusieurs tables.
INSERT	Crée un enregistrement.
UPDATE	Modifie les enregistrements.
DELETE	Supprime les enregistrements.

LCD - Langage de contrôle de données

Commande	Description
GRANT	Donne un privilège à l'utilisateur.
REVOKE	Reprend les privilèges accordés par l'utilisateur.

Présentation et historique :

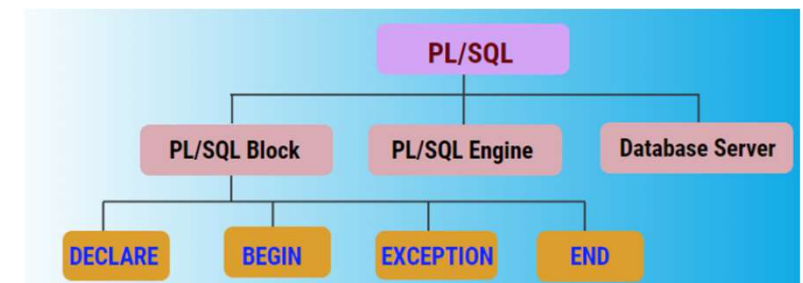
- ❑ **PL/SQL** (*Procedural Langage/Structured Query Langage*) est un **langage procédural** d'Oracle Corporation **étendant SQL** ; il est disponible dans Oracle Database (depuis la version 7). Il permet de **combiner** les avantages d'un **langage de programmation classique** qui offre les structures algorithmiques (les instructions conditionnelles et répétitives, les sous-programmes...) avec les possibilités de **manipulation de données** offertes par SQL.
- ❑ Le langage PL/SQL est utilisé à partir de l'environnement **SQL*Plus** et par les **outils d'Oracle** (Forms, Report et Graphics). Il sert à programmer des procédures, des fonctions, des triggers, et donc plus généralement, des packages.
- ❑ PL/SQL a été développé par Oracle à la fin des années 1980. À l'origine, PL/SQL avait des capacités limitées. Mais cela a changé au début des années 1990.
- ❑ Le langage PL/SQL a évolué avec l'avènement des concepts orientés objet dans **Oracle 8i** et **9i**. PL/SQL n'est plus un langage purement procédural. Il est maintenant à la fois un **langage de programmation procédural** et **orienté objet**.
- ❑ À partir de la **version 11g d'Oracle**, PL/SQL a changé d'un langage interprété en **langage compilé**.
- ❑ PL/SQL est un **langage structuré en bloc**. Il n'interprète pas une commande mais un ensemble de commandes contenu dans un bloc. Ce bloc est **compilé** et **exécuté** par le **moteur de PL / SQL**.

Généralités sur PL/SQL



Moteur PL/SQL :

- ❑ Le langage PL/SQL est un **outil robuste** qui permet **d'écrire des programmes** et de les déployer dans une base de données. Il peut **simplifier** le développement des applications, **optimiser** l'exécution, et **améliorer** l'utilisation et l'exploitation des ressources dans la base de données.
- ❑ Les instructions SQL et PL/SQL sont regroupées **en blocs**, qui sont utilisés par le serveur et par certains outils Oracle. Ils sont reçus et exécutés par le **moteur PL/SQL** (PL/SQL Engine), qui peut se trouver soit dans l'outil, soit dans le serveur Oracle.
- ❑ Le moteur accepte en entrée une unité valide ; il traite chaque instruction PL/SQL et sous-traite les instructions purement SQL au **moteur SQL**, afin de réduire le trafic réseau.
- ❑ Lorsque le moteur PL/SQL reçoit un bloc pour exécution, il effectue les opérations suivantes :
 - **séparation** des ordres SQL et PL/SQL ;
 - passage des **commandes SQL** au **processeur SQL** (*SQL statement executor*) ;
 - passage des instructions procédurales au processeur d'instructions procédurales (*procedural statement executor*).
- ❑ Ce mode de fonctionnement a pour effet de **minimiser la charge** du serveur Oracle et de **réduire** le nombre de curseurs nécessaires en mémoire.



Différence entre SQL et PL/SQL



Voici quelques différences importantes entre [SQL](#) et [PL/SQL](#):

SQL	PL / SQL
SQL est une requête unique utilisée pour effectuer des opérations DML et DDL.	PL/SQL est un bloc de codes utilisé pour écrire l'intégralité des blocs/procédures/fonctions du programme, etc.
Il est déclaratif et définit ce qui doit être fait plutôt que la manière dont les choses doivent être faites.	PL/SQL est une procédure qui définit comment les choses doivent être faites.
Exécuté comme une seule instruction.	Exécuter dans son ensemble.
Principalement utilisé pour manipuler des données.	Principalement utilisé pour créer une application.
Interaction avec un serveur de base de données.	Aucune interaction avec le serveur de base de données.
Ne peut pas contenir de code PL/SQL.	Il s'agit d'une extension de SQL, de sorte qu'elle peut contenir du SQL à l'intérieur.



Qu'est-ce qu'un administrateur de base de données (DataBase Administrator)



- ❑ Un **Administrateur de base de données** ou **DataBase Administrator** est la **personne chargée de maintenir un environnement de ce type**. La **conception**, l'**implémentation**, la **maintenance** du système et la **mise en place de règles**. Il doit aussi **former** les employés de l'entreprise à la **gestion** et à l'**utilisation** de la **BDD**.
- ❑ En règle générale, un **DBA dispose d'une formation dans le domaine des sciences informatiques** et d'une expérience professionnelle avec une spécifique ou diverses bases de données. Il doit aussi avoir une expérience avec les principaux produits de gestion de database comme **SQL**, **SAP** ou **Oracle**.



Le rôle d'un administrateur de base de données ?



- ❑ Les entreprises ont besoin de stocker des informations importantes pour leur fonctionnement quotidien. Un administrateur de base de données, aussi appelé **Database Administrator (DBA)**, doit être capable de travailler avec différents types de bases de données :
 - les **bases de données relationnelles** ;
 - les **bases de données NoSQL** ;
 - les **bases de données en mémoire (IMDB)** (L'Internet Movie Database, abrégé en IMDb, est une base de données en ligne sur le cinéma mondial, sur la télévision, et plus secondairement les jeux vidéo)
- ❑ L'administrateur de base de données doit également être capable de **comprendre les besoins** de l'entreprise en matière de **stockage** de données et de **s'assurer** que les données sont **stockées de manière efficace et sécurisée**. Il doit être en mesure de **surveiller la performance** du système et de **détecter les problèmes** potentiels avant qu'ils ne deviennent critiques.



Le rôle d'un administrateur de base de données ?



- ❑ Un des challenges de l'administrateur ou administratrice de base de données est de permettre aux utilisateurs **l'accès aux informations** à tout moment. Pour ce faire, le **DBA** met en place des **procédures**, des **droits d'accès**, des **normes d'utilisation** des systèmes et **reste à l'écoute** des retours pour **améliorer** l'expérience des interactions avec les données. Cela peut inclure la mise en place de **systèmes de rapports** et de **requêtes** pour aider les utilisateurs à trouver les informations dont ils ont besoin rapidement et facilement.
- ❑ Ce professionnel travaille en **étroite collaboration** avec d'autres membres de l'équipe informatique pour **s'assurer** que les **systèmes de bases de données** sont bien **intégrés** avec d'autres applications utilisées par l'organisation. Cela peut inclure des applications de gestion de la relation client ([CRM](#)), des systèmes de **gestion de projet** et des **logiciels de comptabilité**.

